BÖLÜM 9 / MODULE 9

Üretim Düzeyinde AI Sistem Tasarımı (MAANG Vaka Çalışmaları)

Production AI System Design: Flagship MAANG Case Studies

Gerçek dünya AI Sistem Tasarımı: Kurumsal Müşteri Destek Ajanı, GitHub Copilot Kod Asistanı, Kurumsal Perplexity/Glean Arama Motoru, Çok Modlu E-Ticaret Öneri Sistemi, Finansal Map-Reduce Raporlama ve OpenAI Realtime Sesli Asistan.

Real-world AI System Design case studies: Enterprise Customer Support Agent, GitHub Copilot Code Assistant, Enterprise Perplexity/Glean Search Engine, Multimodal E-Commerce Recommender, Financial Map-Reduce Summarizer, and OpenAI Realtime WebRTC Voice Assistant.

49

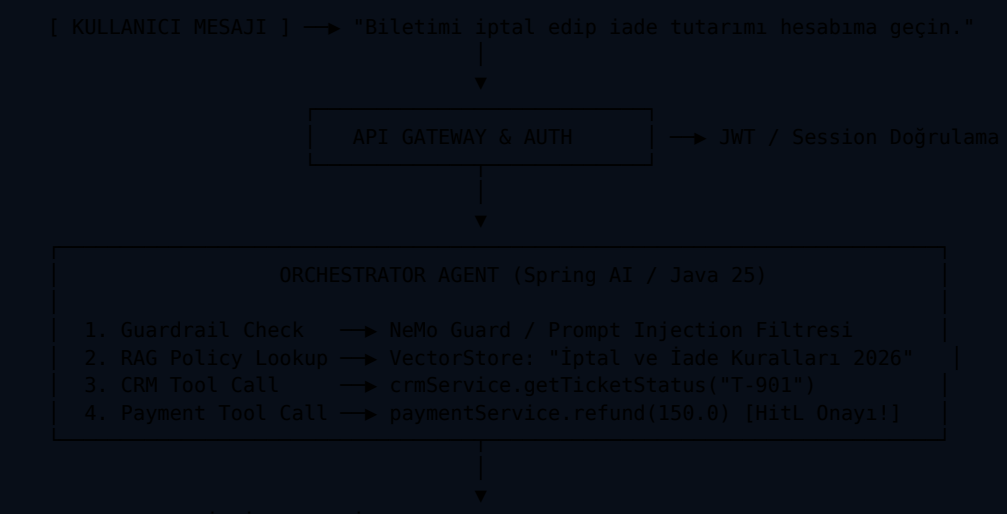
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 1: Kurumsal Müşteri Destek Ajanı Tasarımı (Multi-Turn, RAG, CRM Tool)

Case 1: Enterprise Customer Support Agent: Multi-Turn Memory, RAG & CRM Tools

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir havaalanındaki müşteri hizmetleri masasına gittiğinizi düşünün. Karşınızdaki görevli hem şirketinizin kural kitapçığını bilir (RAG - 'Uçuştan 2 saat önce bilet iptal edilebilir'), hem bilgisayarındaki ekrandan sizin biletinizi ve ödemenizi görür (CRM & Tool Calling), hem de sizinle 10 dakikadır ne konuştuğunuzu hatırlar (Multi-Turn Bellek). Görevli kafasına göre para vermez; bilgisayara tıklar, banka sistemini tetikler ve paranızı kartınıza iade eder. MAANG mülakatında bir destek ajanı tasarlarken işte bu canlı müşteri masasının tüm bilgisayar sistemlerini ve güvenlik duvarlarını çizmeniz gerekir!

🇬🇧 ENGLISH (ELI5)

Imagine stepping up to an airline customer service desk. The representative knows company refund bylaws by heart (RAG), pulls up your reservation on their terminal (CRM & Database Tools), tracks your conversation seamlessly (Multi-Turn Memory), and triggers the credit card refund with supervisor authorization (Human-in-the-Loop Tool). Designing an enterprise AI Support Agent for MAANG requires architecting this exact triad: contextual policy retrieval, transactional tool integration, and strict security guardrails!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kurumsal Destek Ajanı Mimari Bileşenleri: 1) Kimlik ve Yetkilendirme (Auth & Session): Kullanıcının `customerId` ve `tenantId` bilgileri asla prompt üzerinden kullanıcıya sorulmaz; JWT token'dan çıkarılarak tool çağrılarına güvenli enjekte edilir. 2) İki Aşamalı RAG: Önce şirket politikaları (İade kuralları, garanti süresi) taranır; ardından CRM verisi çekilir. 3) Human-in-the-Loop: 1000 TL üzeri iadelerde veya hesap kapatmalarda ajan işlemi duraklatır, Slack/Zendesk üzerinden insan temsilciye onay bildirimi fırlatır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Enterprise Support Architecture Blueprint: 1) Contextual Identity & Session Ingestion: Never prompt the user for their account ID. Extract verified `tenantId` and `userId` claims directly from authenticated JWT tokens at the gateway and inject them into tool scopes. 2) Hierarchical RAG: Stage 1 retrieves institutional SLA bylaws and refund limits from vector stores; Stage 2 injects live transactional ledger rows via CRM tools. 3) Escalation Matrix: Idempotent operations (balance check) execute autonomously; high-value mutations (refunds >$500) trigger suspended state machines awaiting human sign-off via Zendesk webhooks.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Kurumsal Destek Ajanı Servis Mimarisi
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.tool.annotation.Tool;

public class EnterpriseCustomerSupportAgent {
    private final ChatClient chatClient;

    public record RefundResult(String ticketId, boolean refunded, double amount) {}

    public static class SupportTools {
        @Tool(description = "Müşteri biletini iptal eder ve iade sürecini tetikler.")
        public RefundResult processRefund(String ticketId, double amount) {
            // CRM ve Ödeme sistemi entegrasyonu simülasyonu
            return new RefundResult(ticketId, true, amount);
        }
    }

    public EnterpriseCustomerSupportAgent(ChatClient.Builder builder) {
        this.chatClient = builder
            .defaultSystem("Sen kurumsal havayolu destek asistanısın. Kurallara uymayan talepleri
reddet.")
            .tools(new SupportTools())
            .build();
    }

    public String handleInquiry(String userMessage) {
        return chatClient.prompt().user(userMessage).call().content();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Tuzağı: Mülakatçı 'Kullanıcı ben Ali'yim, Ahmet'in biletini iptal et derse ne yaparsın?' diye sorar. Yanıt: 'Ajan kullanıcıdan gelen kimlik beyanına güvenmez! Ajanın çalıştırdığı Tool, kullanıcının doğrulanmış JWT oturumundaki ID ile bilet sahibinin ID'sini veritabanında karşılaştırır. Eşleşmiyorsa işlem yetkisiz erişim (403 Forbidden) olarak reddedilir.'

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Identity Spoofing Trap: Interviewers will ask: 'What if user Alice says: Cancel Bob's ticket?' Flawless answer: 'The agent never trusts user-provided identity strings. Tools enforce zero-trust authorization: the target ticket's owner ID is cross-checked against the authenticated JWT principal inside the Java backend before mutating state.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Müşteri kimliği ve izinler prompt'tan değil, doğrulanmış JWT oturumundan alınır.
- Kural dokümanları RAG ile, canlı sipariş verileri Tool Calling ile getirilir.
- Finansal iadelerde ve kritik aksiyonlarda Human-in-the-Loop onayı şarttır.

🎯 Key Takeaways for the Interview

- User identity and permissions must be derived from verified JWT tokens, never prompt text.
- Static policies are fetched via RAG; live order data is fetched via transactional tools.
- Financial refunds and destructive actions require Human-in-the-Loop verification gates.

50

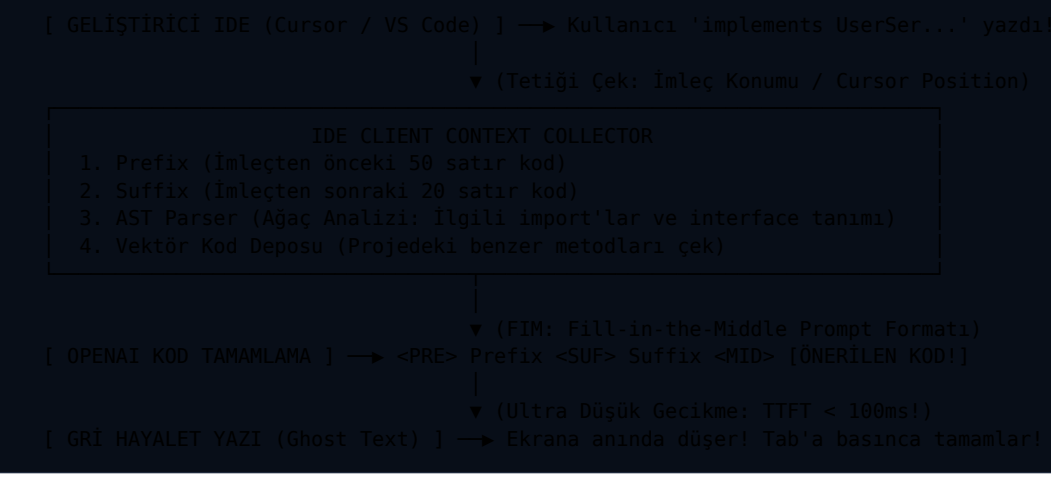
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 2: Kod Asistanı Mimarisi (GitHub Copilot Klonu: AST, Vector & Streaming)

Case 2: Code Assistant Architecture: AST Parsing, Vector Code Indexing & Fill-In-The-Middle

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Akıllı bir çırak marangozun yanınızda durduğunu düşünün. Siz tahtayı ölçüp tam 'Bana çiv...' dediğiniz anda, çırak elinize çekici ve doğru boydaki çiviyi uzatır (Siz cümleyi bitirmeden ihtiyacınızı anlar!). GitHub Copilot tam olarak bu çıraktır: İmlecinizin nerede olduğuna bakar, yukarıda hangi sınıfları import ettiğinizi inceler (AST), projenin diğer dosyalarında nasıl kod yazdığınızı hatırlar ve siz daha klavyeye basarken ekrana açık gri renkte doğru kodu fısıldar!

🇬🇧 ENGLISH (ELI5)

Imagine a brilliant apprentice standing beside a master surgeon. Before the surgeon even finishes articulating the request, the apprentice slaps the exact scalpel size into the surgeon's palm. A real-time code assistant operates with this predictive foresight: it captures the cursor coordinates, parses the Abstract Syntax Tree (AST) of the active workspace, matches relevant library imports, and leverages Fill-in-the-Middle (FIM) prompt formatting to project ghost text within 100 milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kod Asistanı Sistem Tasarımı Gereksinimleri: 1) FIM (Fill-in-the-Middle): Standart LLM sadece sonrasını tahmin eder. Kod asistanı ise imlecin üstündeki kod (Prefix) ile altındaki kodu (Suffix) modele verir ve aradaki boşluğu (\<MID>) doldurmasını ister. 2) AST (Abstract Syntax Tree) Analizi: Düz metin aramak yerine Tree-Sitter kütüphanesiyle kod ağacı çıkarılır; çağrılan fonksiyonun interface tanımı ve import'ları bağlama eklenir. 3) Katı Gecikme SLA'sı: Geliştirici yazarken önerinin çıkması için TTFT < 150ms olmak zorundadır! Bu yüzden küçük, kod için optimize edilmiş hızlı modeller (GPT-4o-mini / StarCoder) kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Code Assistant Engineering & Latency Budgets: 1) Fill-in-the-Middle (FIM): Specialized models ingest three partitioned segments: \<PRE> (pre-cursor text), \<SUF> (post-cursor text), and generate within \<MID>. This enables context-aware autocompletion inside existing method bodies. 2) Tree-Sitter AST Slicing: Rather than dumping whole files, a local daemon parses AST nodes to extract active class hierarchies, interface contracts, and imported method signatures. 3) Sub-150ms SLA: Programmers type at 5-10 characters/sec. If ghost text latency exceeds 150ms, the developer has already typed past the completion point, rendering it useless. Speculative decoding and aggressive client-side debouncing (50ms) are non-negotiable.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Kod Asistanı için Fill-In-The-Middle (FIM) Prompt Modeli
public record FimCodePrompt(String filePath, String prefix, String suffix) {
    public String toOpenAiFimFormat() {
        return String.format("""
            // Dosya: %s
            <PRE>
            %s
            <SUF>
            %s
            <MID>
            """, filePath, prefix, suffix);
    }
}

void main() {
    var prompt = new FimCodePrompt(
        "OrderService.java",
        "public class OrderService {
    public void cancelOrder(String id) {",
        "    }
}"
    );

    IO.println("Copilot FIM Prompt Yapısı:");
    IO.println(prompt.toOpenAiFimFormat());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Telif Hakkı ve Güvenlik Riski: Model internetten ezberlediği lisanslı (GPL) veya zararlı bir kod parçasını geliştiricinin projesine doğrudan önerebilir. Kurumsal kod asistanlarında üretilen kodun açık kaynak kütüphanelerle birebir eşleşip eşleşmediğini kontrol eden bir 'License & Secret Scanner' filtresi bulunmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Copyright & Secret Leakage: Generative code models risk spitting out memorized GPL-licensed code or accidentally echoing hardcoded AWS API tokens found in training sets. Enterprise code assistants must integrate a real-time post-generation license and secret scanner before displaying ghost text.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Fill-in-the-Middle (FIM), kodun hem öncesini hem sonrasını görerek arayı doldurur.
- AST (Tree-Sitter) analizi, sadece imlecin etrafındaki ilgili tipleri ve metotları seçer.
- Kod asistanlarında gecikme SLA'sı (TTFT \< 150ms) başarıyı belirleyen 1 numaralı faktördür.

🎯 Key Takeaways for the Interview

- Fill-in-the-Middle (FIM) generates code aware of both prefix and suffix boundaries.
- AST parsing isolates relevant imports, interfaces, and types to keep context razor-sharp.
- Sub-150ms TTFT latency is the defining engineering SLA for real-time code completion.

51

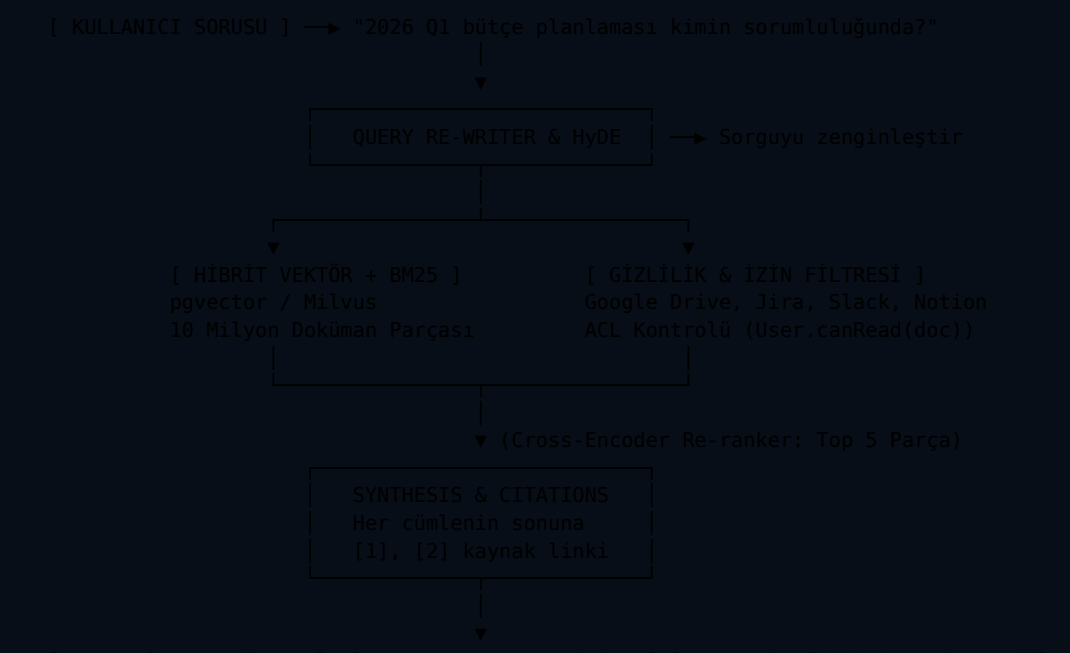
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 3: Milyon Dokümanlı Kurumsal Arama Motoru (Glean / Perplexity Klonu)

Case 3: Enterprise Search Engine: Glean & Perplexity Architecture at Scale

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Şirketinizde 10.000 çalışan olduğunu ve herkesin Google Drive'a, Slack'e, Jira'ya ve Notion'a her gün yüzlerce dosya attığını düşünün. 'Bizim şirketin yeni ofis adresi neydi?' diye aradığınızda 50 farklı eski adres çıkar. Glean ve Perplexity gibi kurumsal arama motorları şirketin tüm hafızasını tarar: En güncel dosyayı bulur, kimin yetkisi olduğunu kontrol eder (Ahmet'in gizli maaş dosyasını Mehmet'e göstermez) ve cevabı tam bir makale gibi yazıp her cümlenin yanına 'Bu bilgi şu linkten alındı [1]' diye tıklanabilir mavi kaynak linkleri koyar!

🇬🇧 ENGLISH (ELI5)

In a 10,000-person tech enterprise, company knowledge is scattered across millions of Google Docs, Jira epics, Slack channels, and Notion pages. Searching for 'What is our corporate remote travel budget?' yields 50 obsolete spreadsheets. An enterprise search engine (like Glean or Perplexity) indexes this multi-system knowledge graph: it enforces user-level Access Control Lists (ACLs), synthesizes a coherent direct answer, and attaches verified, clickable footnote citations (`[1]`, `[2]`) to every assertion!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kurumsal Perplexity Mimarisi: 1) ACL (Access Control Lists) Senkronizasyonu: En kritik zorluktur. Dokümanlar vektör veritabanına yazılırken Google Drive/Slack izin grupları (`allowed_users`, `allowed_roles`) metadata olarak işlenir. Arama anında kullanıcının yetkisi olmayan hiçbir doküman sorgu havuzuna giremez. 2) Çift Aşamalı Getirme + Cross-Encoder: 10 milyon dokümandan hibrit arama ile 100 aday çekilir; Cohere Rerank ile en iyi 5 parçaya indirgenir. 3) Kaynak Doğrulama (Citation Attributor): Model her iddiayı bir doküman ID'si ile eşleştirmek zorundadır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Enterprise Cognitive Search System Design: 1) Dynamic ACL Ingestion: Document ingestion pipelines must ingest upstream permission graphs (Google Workspace, Confluence, Slack ACLs). When a user searches, the vector store pre-filters documents against the user's verified security groups. 2) Two-Stage Pipeline with Dense/Sparse Fusion: Scans 10M+ documents via Elastic/BM25 and HNSW, merging candidates via RRF and pruning to top-5 through cross-encoders. 3) Footnote Citation Engine: Output synthesis requires inline citations (`[1]`). A post-processing AST validator checks that every referenced citation index genuinely contains the cited fact.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Doğrulanmış Kaynak Gösterimi (Citation Attribution) Modeli
import java.util.List;

public class EnterpriseSearchResult {
    public record Citation(int index, String documentTitle, String sourceUrl, String snippet) {}
    public record SynthesizedAnswer(String answerText, List<Citation> citations) {}

    public static SynthesizedAnswer generateMockResponse() {
        var citation1 = new Citation(1, "2026 Q1 Bütçe Planı", "https://drive.company.com/doc/101",
"Planlama sorumlusu: Merve Kaya.");
        var citation2 = new Citation(2, "Şirket Organizasyon Şeması", "https://notion.company.com/
org", "Merve Kaya Finans Direktörüdür.");

        String text = "2026 yılı Q1 bütçe planlaması Finans Direktörü Merve Kaya'nın sorumluluğundadır
[1]. Merve Hanım doğrudan CFO'ya raporlar [2].";
        return new SynthesizedAnswer(text, List.of(citation1, citation2));
    }
}

void main() {
    var searchResult = EnterpriseSearchResult.generateMockResponse();
    IO.println("Kurumsal Arama Yanıtı:");
    IO.println(searchResult.answerText());
    IO.println("
Doğrulanmış Kaynaklar:");
    for (var c : searchResult.citations()) {
        IO.println("[" + c.index() + "] " + c.documentTitle() + " -> " + c.sourceUrl());
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Güvenlik İhlali (Permission Leakage): Ajan Ahmet'in sorusunu yanıtlarken yetkisiz olduğu bir belgeden veri çeker ve cevabı ekrana yazarsa, belgeye link vermese bile gizli bilgi sızmış olur! İzin kontrolü model aşamasında DEĞİL; veritabanı vektör sorgusu aşamasında (WHERE allowed_users CONTAINS) yapılmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Permission exfiltration disaster: If permission filtering occurs after retrieval, an unauthorized document summary can leak confidential salary figures in the final synthesis. Access Control Lists must be evaluated as strict pre-filters directly at the database index layer.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kurumsal aramada erişim izinleri (ACL) modelde değil, veritabanı aramasında filtrelenmelidir.
- Kaynak gösterimi (Citations), kurumsal güven ve doğrulanabilirlik için zorunludur.
- Hibrit arama + Re-ranking, milyonlarca dokümandan en güncel cevabı çekmenin tek yoludur.

🎯 Key Takeaways for the Interview

- Access control lists (ACLs) must be enforced at the database layer, not within prompt text.
- Verified footnote citations are mandatory for enterprise trust and regulatory compliance.
- Hybrid search coupled with cross-encoder re-ranking is essential to cut through corporate noise.

52

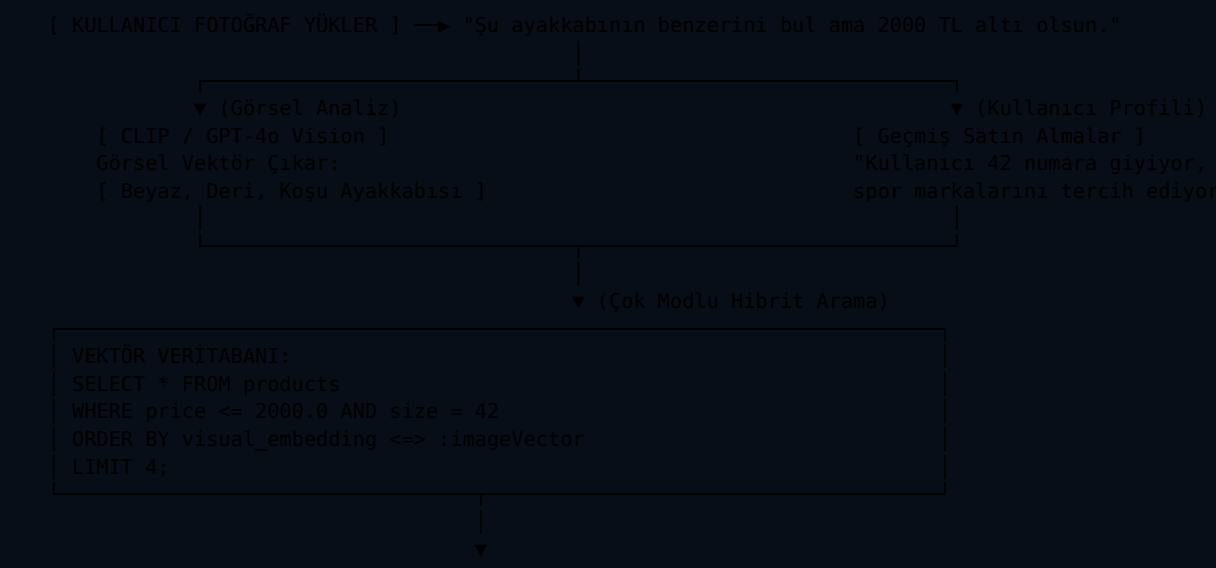
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 4: Çok Modlu E-Ticaret Öneri Ajanı (Görsel Arama + Kullanıcı Geçmişi)

Case 4: Multimodal E-Commerce Recommender: Visual Search & Collaborative Personalization

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Sokakta yürürken birinin ayağında çok şık bir spor ayakkabı gördünüz ama markasını bilmiyorsunuz. Hemen telefonunuzla fotoğrafını çekip e-ticaret uygulamasına atıyorsunuz. Uygulama resme bakıyor, 'Bu beyaz deri bir koşu ayakkabısı' diyor (Görsel Arama). Sonra sizin daha önceki siparişlerinize bakıyor: 'Bu müşteri 42 numara giyiyor ve 2000 TL'den fazla para vermek istemiyor.' Sonuçta karşınıza hem gördüğünüz ayakkabıya ikiz gibi benzeyen, hem tam ayağınıza göre olan, hem de cebinizi yakmayan 3 model çıkarıyor!

🇬🇧 ENGLISH (ELI5)

You spot someone wearing stylish sneakers on the street, but you don't know the brand. You snap a quick photo and upload it to an e-commerce app. The multimodal vision model analyzes the visual features: white leather, low-top running profile (Visual Search). The recommender intersects this with your historical profile: 'Size 10, prefers breathable athletic soles, price ceiling $100.' Within 200ms, the system returns three visually identical pairs in your exact size and budget!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Çok Modlu Öneri Mimarisi: 1) Multimodal Embeddings (CLIP / ImageBind): Resimler ve metinler aynı vektör uzayına izdüşürülür. Böylece bir resim vektörü ile 'beyaz deri ayakkabı' metin vektörü arasındaki mesafe doğrudan ölçülebilir. 2) İki Vektörün Füzyonu: Ürün metin vektörü ile ürün görsel vektörü ağırlıklı olarak birleştirilir (V_final = 0.6 · V_image + 0.4 · V_text). 3) Dinamik Filtreleme: Fiyat, stok durumu ve beden bilgileri SQL WHERE koşulu ile elenir; stokta olmayan ürün kullanıcıya önerilmez.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Multimodal E-Commerce Architecture: 1) Joint Embedding Space (CLIP / OpenCLIP): Projects visual features and textual tokens into a shared semantic manifold. An image vector can be directly queried with natural language predicates. 2) Multimodal Feature Fusion: Composite embeddings blend visual and textual representations (V_final = α · V_visual + (1-α) · V_text). 3) Hard Metadata Gating: Pre-filter out-of-stock items, size mismatches, and price ceiling breaches via SQL predicates prior to vector traversal, ensuring all presented recommendations are immediately purchasable.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Çok Modlu E-Ticaret Arama ve Filtreleme Modeli
public record ProductRecommendation(String sku, String title, double price, int size, double
visualMatchScore) {}

public class MultimodalProductSearchService {
    public static List<ProductRecommendation> rankProducts(byte[] userUploadedImage, int userShoeSize,
double maxPrice) {
        // Görsel embedding ve veritabanı filtreleme simülasyonu
        return List.of(
            new ProductRecommendation("SKU-941", "UltraRide Beyaz Spor Ayakkabı", 1850.0,
userShoeSize, 0.94),
            new ProductRecommendation("SKU-812", "ProRunner Deri Sneaker", 1990.0, userShoeSize, 0.89)
        );
    }
}

void main() {
    var recommendations = MultimodalProductSearchService.rankProducts(new byte[0], 42, 2000.0);
    IO.println("Kullanıcıya Özel Çok Modlu Öneriler:");
    for (var p : recommendations) {
        IO.println("• " + p.title() + " | Fiyat: " + p.price() + " TL | Eşleşme: %" + (int)
(p.visualMatchScore() * 100));
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Stok ve Fiyat Tuzağı: Kullanıcıya görsel olarak birebir benzeyen bir ürün önerip, kullanıcı 'Satın Al'a tıkladığında 'Stokta yok' veya 'Fiyatı 10.000 TL' derseniz müşteri kaybedersiniz. Vektör aramasında stok durumu (`stock > 0`) DAİMA zorunlu bir ön filtre olmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Ghost inventory trap: Recommending a visually identical product that is out-of-stock or vastly outside the user's budget destroys conversion rates. Always enforce hard pre-filtering on `inventory_count > 0` before computing nearest neighbors.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CLIP ve multimodal modeller, resimlerle metinleri aynı vektör uzayında buluşturur.

• Görsel benzerlik ile kişiselleştirilmiş kullanıcı geçmişi ağırlıklı olarak birleştirilmelidir.

• Stok ve fiyat filtreleri vektör aramasından önce (Pre-filter) kesin olarak uygulanmalıdır.

🎯 Key Takeaways for the Interview

• Multimodal models map images and text into a unified, cross-searchable vector space.

• Visual similarity must be fused with user purchase history and size preferences.

• Stock availability and price ceilings must be enforced as non-negotiable database pre-filters.

53

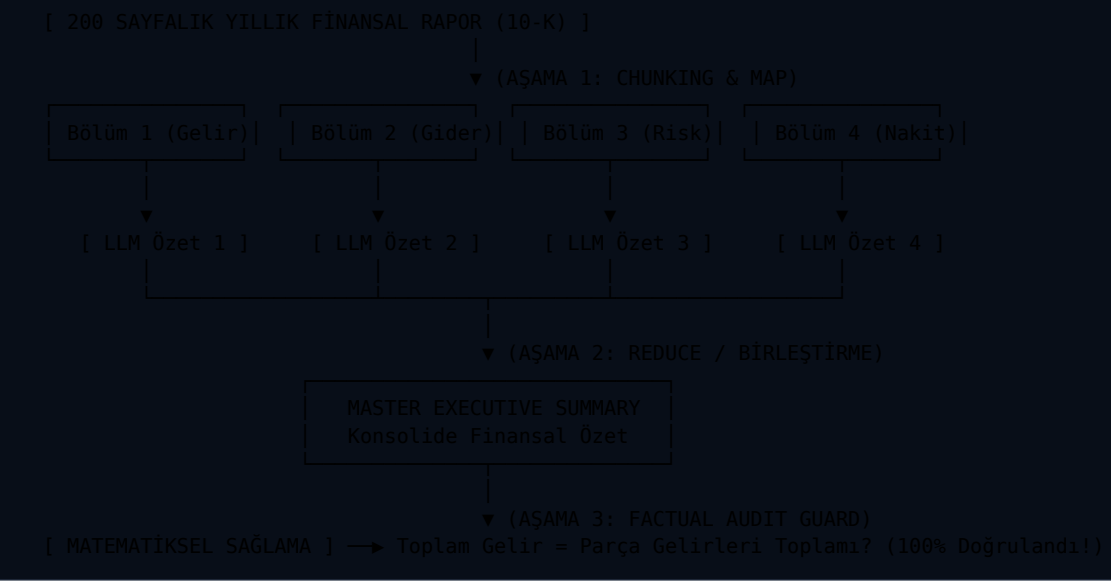
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 5: Finansal Rapor Analizi & Özetleme (Chunking, Map-Reduce & Guard)

Case 5: Financial Report Summarization: Map-Reduce Chunking & Fact Extraction Guards

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

200 sayfalık kalın bir şirket bilançosunu tek bir kişinin 5 dakikada okuyup özetlemesini bekleyemezsiniz; ya önemli bir borcu atlar ya da rakamları karıştırır. Bunun yerine raporu 4 kişiye paylaştırırsınız (Map adımı): Biri sadece gelirleri, biri giderleri, biri borçları, biri de riskleri 1 sayfada özetler. Sonra genel müdür bu 4 özeti önüne alır ve tek bir 1 sayfalık 'Yönetici Özeti' çıkarır (Reduce adımı). En son da hesap makinesiyle rakamları toplar: 'Özetlerdeki sayılar birbirini tutuyor mu?' (Hallucination Guard). Finansal yapay zeka sistemleri Map-Reduce ile çalışır!

🇬🇧 ENGLISH (ELI5)

You don't expect a single financial analyst to ingest a 300-page SEC 10-K annual filing in 5 minutes without missing liabilities or hallucinating debt figures. You deploy a distributed team of 4 junior analysts (Map phase): Analyst 1 summarizes revenues, Analyst 2 audits expenditures, Analyst 3 details operational risks, and Analyst 4 tracks balance sheets. The CFO combines the 4 summaries into a unified 1-page executive brief (Reduce phase). A computational auditor verifies that the balance sheet numbers tally across the summary (Hallucination Guard)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Büyük Doküman Özetleme Stratejileri: 1) Stuffing: Tüm metni tek prompt'a basmak (Sadece küçük dokümanlar için geçerlidir; 200 sayfada çöker). 2) Map-Reduce Deseni: Doküman mantıksal bölümlere ayrılır; her bölüm bağımsız bir prompt ile özetlenir (Map). Tüm alt özetler birleştirilerek nihai yönetici özeti çıkarılır (Reduce). Java 25 Virtual Threads ile 50 bölüm paralel özetlenir! 3) Refine Deseni: Sıralı olarak önceki özet yeni bölümle birleştirilerek güncellenir. 4) Hallucination Guard: Finansal tablolardaki sayısal veriler regex ile çekilir; modelin uydurduğu sayılar orijinal metinle matematiksel olarak karşılaştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Large Document Summarization Topologies: 1) Map-Reduce Architecture: Essential for documents exceeding context windows or suffering from attention dilution. Chunks execute through concurrent Map prompts across Java 25 Virtual Threads to produce intermediate bullet summaries. A second Reduce prompt consolidates intermediate bullets into a cohesive executive report. 2) Refine Loop: Iteratively rolls context through sequentially updated summaries (higher coherence, but serialized and slow). 3) Factual Consistency Guard: Automatically cross-audits every numeric entity (currency, percentages, dates) generated in the summary against a deterministic AST extracted from the source filing.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Map-Reduce Finansal Doküman Özetleme Orkestratörü
import java.util.List;

public class FinancialMapReduceOrchestrator {
    public static String mapSection(String sectionTitle, String rawText) {
        // Alt bölümü bağımsız özetleme simülasyonu (Virtual Thread içinde paralel çalışır)
        return "[" + sectionTitle + " Özeti]: Gelir %14 artışla 4.2 Milyar $'a ulaştı.";
    }

    public static String reduceSummaries(List<String> sectionSummaries) {
        // Tüm alt özetleri tek bir master raporda birleştirme
        return "2026 YILLIK FİNANSAL YÖNETİCİ ÖZETİ:
" + String.join("
", sectionSummaries);
    }
}

void main() {
    var summaries = List.of(
        FinancialMapReduceOrchestrator.mapSection("Gelir Tablosu", "Gelir metni..."),
        FinancialMapReduceOrchestrator.mapSection("Nakit Akışı", "Nakit metni...")
    );

    String finalReport = FinancialMapReduceOrchestrator.reduceSummaries(summaries);
    IO.println(finalReport);
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Kritik Finansal Hata: LLM'ler rakamları çarpma ve bölmede berbattır! 'Şirketin kar marjı %12.4'tür' gibi matematiksel hesaplamaları ASLA modele yaptırmayın. Sayısal hesaplamaları Java/Python kodunda yapın; modeli sadece bu kesin sayıları güzel bir metin haline getirmek için kullanın.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Mathematical hallucination warning: LLMs are probabilistic language models, not arithmetic calculators. Never prompt an LLM to multiply or divide balance sheet numbers. Compute all financial KPIs deterministically in Java code, feeding the pre-calculated numbers to the model strictly for stylistic narrative presentation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Yüzlerce sayfalık büyük dokümanlarda Map-Reduce mimarisi dikkat kaybını önler.
- Java 25 Virtual Threads, onlarca bölümün aynı anda paralel özetlenmesini sağlar.
- Finansal sayılar model tarafından hesaplanmamalı; Java kodunda hesaplanıp modele verilmelidir.

🎯 Key Takeaways for the Interview

- Map-Reduce summarization prevents context degradation across massive 100+ page filings.
- Java 25 Virtual Threads execute dozens of Map tasks concurrently with minimal overhead.
- Deterministic financial arithmetic must be computed in code, not delegated to LLMs.

54

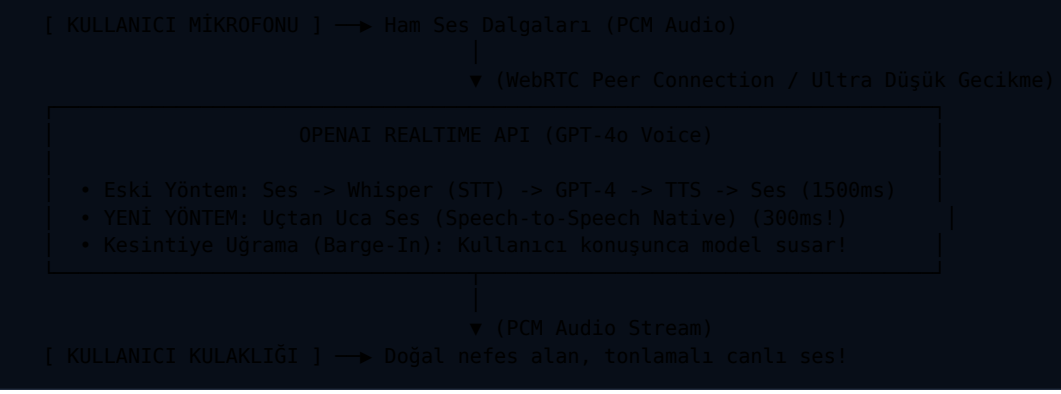
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 6: Gerçek Zamanlı Sesli AI Asistanı (OpenAI Realtime API, WebRTC & Ses)

Case 6: Real-Time Voice AI: OpenAI Realtime API, WebRTC & Speech-to-Speech

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski sesli asistanları (Siri veya eski çağrı merkezleri) düşünün: Siz konuşurdunuz, sistem 3 saniye sessiz kalır, önce söylediklerinizi yazıya döker (STT), sonra cevabı düşünür, sonra o cevabı bir robota okuturdu (TTS). Konuşma robotik ve kopuk olurdu. OpenAI Realtime API ise canlı bir insanla telefonda konuşmak gibidir! Arada hiçbir yazıya çevirme beklemesi yoktur; ses dalgaları doğrudan modelin beynine girer ve 300 milisaniyede doğal bir insan sesiyle anında cevap döner. Siz araya girip 'Dur bir dakika' dediğiniz anda model anında susar ve sizi dinler (Barge-In)!

🇬🇧 ENGLISH (ELI5)

Legacy voice assistants suffered from the clumsy 3-hop cascade: Speech-to-Text (Whisper) -> LLM Text Synthesis (GPT-4) -> Text-to-Speech (Polly). This introduced a 2,000ms pause, stripping away human emotional cadence. OpenAI's native Realtime API communicates via WebRTC Speech-to-Speech: raw audio enters the neural network and synthesized speech returns in under 300ms! Crucially, it supports native interruption (Barge-In): the instant the user speaks, the assistant immediately ceases transmission and listens!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Realtime Ses Mimarisi Mühendisliği: 1) WebRTC Protokolü: WebSocket'e kıyasla UDP tabanlı WebRTC MediaStream gecikmeyi 200-300ms seviyesine çeker (Gerçek zamanlı insan konuşma eşiği). 2) Speech-to-Speech (Uçtan Uca Ses): Model sadece metin değil, kullanıcının ses tonunu, heyecanını, nefes alışını ve fısıltısını anlar; aynı tonlamayla cevap verir. 3) Server VAD (Voice Activity Detection): Model kullanıcının ne zaman susup ne zaman konuşmaya başladığını otomatik algılar; kullanıcı araya girdiğinde (Barge-In) model ses üretimini anında keser (Audio buffer truncate). 4) Realtime Function Calling: Konuşma sırasında arka plandaki Java API'lerini ses gecikmesi yaşamadan tetikleyebilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Speech-to-Speech Realtime Architecture: 1) WebRTC PeerConnections: Bypasses TCP head-of-line blocking by streaming raw 24kHz PCM audio frames over UDP channels, hitting sub-300ms end-to-end latency. 2) Native Auditory Attention: The model attends directly to acoustic spectrogram features, comprehending prosody, emotional inflection, sarcasm, and background interruptions. 3) Server-Side Voice Activity Detection (VAD): Distinguishes ambient noise from genuine speech. Triggers instant audio output truncation when a user interrupts (Barge-In). 4) In-Band Function Calling: Emits structured tool calls concurrently without terminating the WebRTC audio transport.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: OpenAI Realtime WebRTC Oturum Başlatıcı Modeli
public record RealtimeSessionConfig(
    String model,
    String voice, // alloy, echo, shimmer
    double temperature,
    String turnDetectionMode // server_vad
) {
    public static RealtimeSessionConfig createDefault() {
        return new RealtimeSessionConfig("gpt-4o-realtime-preview", "alloy", 0.7, "server_vad");
    }
}

void main() {
    var config = RealtimeSessionConfig.createDefault();
    IO.println("=== OpenAI Realtime API WebRTC Yapılandırması ===");
    IO.println("Model: " + config.model());
    IO.println("Ses Karakteri: " + config.voice());
    IO.println("Konuşma Algılama (VAD): " + config.turnDetectionMode());
    IO.println("Gecikme Hedefi: <300ms (İnsan seviyesi akıcı konuşma)");
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Maliyet Uyarısı: OpenAI Realtime ses API'si metin API'sinden yaklaşık 10 kat daha pahalıdır (Dakikası yaklaşık 0.06 - 0.20 $). Boşta açık kalan WebRTC oturumları faturayı şişirir. Kullanıcı 30 saniye konuşmadığında oturum otomatik olarak kapatılmalıdır (Idle Disconnect).

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Cost vulnerability: Realtime Speech-to-Speech streaming costs 10x more than standard text tokens ($0.10-$0.25 per minute). Abandoned WebRTC connections running on silent microphones burn budget continuously. Implement an aggressive 30-second idle disconnect timeout.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• WebRTC ve Speech-to-Speech, sesli yapay zekayı 300ms gecikmeyle gerçek zamanlı yapar.

• Server VAD ve Barge-In, kullanıcının asistanın sözünü kesebilmesini (Doğal sohbet) sağlar.

• Realtime API'de boşta kalan bağlantılar için mutlaka Idle Timeout mekanizması kurulmalıdır.

🎯 Key Takeaways for the Interview

• WebRTC and Speech-to-Speech deliver human-grade conversational voice in sub-300ms.

• Server-side VAD and Barge-In enable natural human interruption dynamics.

• Idle timeout policies are mandatory to avoid runaway billing on open audio streams.